Zero to Hero Study Handbook: Local Research Assistant

This handbook is built from static analysis of the repository files under `local-research-assistant/`. It explains the real architecture, real code paths, and real data contracts used in this project.

Module 1: Foundations & Architecture

1.1 What This Project Does

Local Research Assistant is a privacy-first, local-first research workspace. It lets a user:

- Create notebooks as research containers.
- Ingest sources from uploads, websites, GitHub repositories, and YouTube transcripts.
- Parse and OCR source content.
- Index document chunks into a vector database.
- Run hybrid search (semantic + lexical).
- Ask grounded questions with citations.
- Generate research outputs (summary, compare, timeline, glossary, entities).
- Generate study artifacts (study guide, flashcards, quizzes).
- Query a lightweight knowledge graph from extracted entities.

Core runtime surfaces:

- Backend API: `apps/api/src/lra_api/main.py`
- Worker pipeline: `apps/api/src/lra_api/workers/tasks.py`
- Frontend web app: `apps/web/src/main.tsx`

1.2 Core Paradigms and Patterns Used Here

Definitions first:

- Layered architecture: The code is split into API routes, schemas, services, data models, and worker tasks. Example folders: `api/`, `schemas/`, `services/`, `db/`, `workers/`.
- Async request handling: FastAPI endpoints and many service methods are `async`, using `httpx.AsyncClient` and `AsyncSession`.
- Background job processing: Heavy ingestion work is queued with Celery (`process_document_upload`) instead of blocking API requests.
- Polyglot persistence: Different storage systems are used for different workloads: Postgres (metadata and history), Qdrant (vectors), Neo4j (relations), MinIO (raw files), Redis (queue/rate limit).
- Hybrid retrieval: Search combines semantic vector retrieval (`QdrantClient`) and lexical SQL full-text search (`to_tsvector` + `plainto_tsquery`), then merges and optional reranks.
- RAG with citation grounding: Answers are generated from retrieved evidence only, and citation objects are returned/stored.
- Schema-first contracts: Pydantic models in `schemas/` define request/response payloads for most endpoints.
- Service-oriented backend: Business logic is in service classes (`IngestionService`, `RetrievalService`, `StudyService`, etc.), not directly in route handlers.

1.3 Architecture and Component Interaction

High-level component roles:

- FastAPI app composes routes, middleware, auth dependencies, metrics, and startup/shutdown hooks.
- Celery worker runs asynchronous ingestion processing.
- Ingestion service parses, chunks, embeds, indexes, and enriches graph data.
- Retrieval service performs hybrid retrieval and grounded answer generation.
- Frontend pages call API endpoints through Axios and show search/chat/study/graph views.

Main flow (ASCII):

```
                    v
[IngestionService.process_document]
     |      |        |       |
     v      v        v       v
[Parser] [OCR]  [Qdrant] [Neo4j]
     |      |      |       |
     +------v------+-------+
            |
            v
[Document chunks + vectors + graph edges]
            |
            v
[RetrievalService hybrid search + RAG]
            |
            v
[Answer + citations -> API -> Frontend]
```

Key architectural notes:

- Upload/import endpoints return quickly with `202 Accepted` and a job id (`IngestionJobResponse`), while indexing runs in background.
- OCR is fallback-driven: parser first, OCR only when parser text is empty and source appears image/scanned.
- Search and chat reuse the same retrieval service, which keeps behavior consistent across endpoints.
- Conversation memory is persisted in relational tables (`chats`, `messages`, `citations`) and exposed by chat session endpoints.

Module 2: Repository Map

This table focuses on files new contributors should understand first.

File/Directory Path	Primary Responsibility	Key Classes/Functions
`pyproject.toml`	Python package metadata, dependencies, test/lint config	N/A
`.env.example`	Environment variable template for all runtime services	N/A
`docker-compose.yml`	Local infrastructure and service orchestration	N/A
`apps/api/src/lra_api/main.py`	FastAPI app entrypoint and middleware	`app`, `metrics_middleware`, `startup_event`, `shutdown_event`, `liveness`
`apps/api/src/lra_api/api/router.py`	API router assembly	`api_router.include_router(...)`
`apps/api/src/lra_api/api/deps.py`	Authentication dependency wiring	`get_current_user`, `oauth2_scheme`
`apps/api/src/lra_api/core/config.py`	Central typed settings loader	`Settings`, `get_settings`, `parse_allowed_origins`
`apps/api/src/lra_api/core/security.py`	Password hashing and JWT helpers	`hash_password`, `verify_password`, `create_access_token`, `create_refresh_token`, `decode_tok`
`apps/api/src/lra_api/core/rate_limit.py`	Redis-backed rate limiting	`RateLimiter.enforce`, `RateLimiter.close`
`apps/api/src/lra_api/db/models/entities.py`	SQLAlchemy ORM entities	`User`, `Notebook`, `Document`, `DocumentChunk`, `Chat`, `Message`, `Citation`, `StudyMaterial`, etc.
`apps/api/alembic/versions/20260628_0001_initial_schema.py`	Initial database schema migration	`upgrade`, `downgrade`
`apps/api/src/lra_api/services/storage/minio_service.py`	Object storage wrapper	`MinioStorageService.ensure_bucke put_raw_document`, `get_raw_docume`
`apps/api/src/lra_api/services/ingestion/pipeline.py`	End-to-end ingestion/indexing pipeline	`IngestionService.ingest_upload`, `process_document`, `_persist_chunk _run_ocr_with_timeout`
`apps/api/src/lra_api/services/ingestion/parsers.py`	File-type parsing into text	`parse_bytes`, `_parse_pdf`, `_parse_docx`, `_parse_excel`, etc.
`apps/api/src/lra_api/services/ingestion/chunking.py`	Text chunking utility	`Chunk`, `chunk_text`

File/Directory Path	Primary Responsibility	Key Classes/Functions
`apps/api/src/lra_api/services/ingestion/connectors.py`	External source connectors	`fetch_website_text`, `fetch_github_repo_snapshot`, `fetch_youtube_transcript`
`apps/api/src/lra_api/workers/celery_app.py`	Celery app configuration	`celery_app`
`apps/api/src/lra_api/workers/tasks.py`	Background task entrypoint	`process_document_upload`
`apps/api/src/lra_api/services/retrieval/service.py`	Hybrid retrieval + grounded answer creation	`RetrievalService.search`, `_semantic_search`, `_lexical_searc` `_merge_scores`, `_rerank`, `_filter_existing_chunks`, `build_answer`
`apps/api/src/lra_api/services/ollama/client.py`	Ollama API client and model routing	`list_models`, `embed`, `generate`, `ocr_image`, `_request_with_retry`
`apps/api/src/lra_api/services/research/service.py`	Research output generation	`summarize`, `compare`, `timeline`, `glossary`, `entities`
`apps/api/src/lra_api/services/study/service.py`	Study artifact generation	`generate`
`apps/api/src/lra_api/services/graph/service.py`	Neo4j graph extraction/query	`extract_relations`, `upsert_document_graph`, `query`, `cl`
`apps/api/src/lra_api/services/monitoring/system.py`	Runtime resource and queue metrics	`system_status`, `_gpu_metrics`
`apps/api/src/lra_api/api/v1/endpoints/documents.py`	Upload/import/OCR/document APIs	`upload_document`, `import_website`, `import_github`, `import_youtube`, `run_ocr`, `get_ingestion_job`
`apps/api/src/lra_api/api/v1/endpoints/search.py`	Search API	`query_search`
`apps/api/src/lra_api/api/v1/endpoints/chat.py`	Chat sessions, messages, streaming RAG	`create_chat_session`, `get_chat_session`, `list_chat_sessions`, `send_message` `rag_answer`, `stream_message`
`apps/api/src/lra_api/api/v1/endpoints/research.py`	Summary/compare/timeline/glossary/entities APIs	`summarize`, `compare`, `timeline`, `glossary`, `entities`
`apps/api/src/lra_api/api/v1/endpoints/study.py`	Study generation + retrieval APIs	`generate_study_assets`, `list_flashcards`, `list_quizzes`
`apps/api/src/lra_api/api/v1/endpoints/graph.py`	Graph query API	`query_graph`
`apps/api/src/lra_api/api/v1/endpoints/notebooks.py`	Notebook CRUD	`create_notebook`, `list_notebooks`, `update_notebook`, `delete_notebook`
`apps/api/src/lra_api/api/v1/endpoints/knowledge.py`	Knowledge organization APIs	folder/collection/bookmark/note/highlight/anno handlers
`apps/api/src/lra_api/api/v1/endpoints/system.py`	Model list and runtime status APIs	`list_models`, `system_status`
`apps/api/src/lra_api/api/v1/endpoints/health.py`	Dependency health checks	`health_check`
`apps/web/src/app/router.tsx`	Frontend route tree and auth gate	`AppRouter`, `ProtectedLayout`
`apps/web/src/services/api.ts`	Frontend HTTP client	`api` Axios instance + auth interceptor
`apps/web/src/stores/auth.ts`	Local auth state store	`useAuthStore`
`apps/web/src/pages/LoginPage.tsx`	Login UI and token storage	`login`
`apps/web/src/pages/DocumentsPage.tsx`	Upload UI	`onUpload`, `handleInput`
`apps/web/src/pages/SearchPage.tsx`	Search UI	`run`
`apps/web/src/pages/ChatPage.tsx`	Chat + SSE streaming UI	`run`
`apps/web/src/pages/StudyToolsPage.tsx`	Study generation UI	`generate`
`apps/web/src/pages/FlashcardsPage.tsx`	Flashcard viewer UI	`load`
`apps/web/src/pages/QuizzesPage.tsx`	Quiz viewer UI	`load`
`apps/web/src/pages/KnowledgeGraphPage.tsx`	Graph query UI	`run`
`docs/api/API.md`	Endpoint index	N/A
`docs/architecture/ARCHITECTURE.md`	Conceptual architecture notes	N/A

Module 3: Core Execution Flows

3.1 App Startup, Routing, and Auth Guard

Step-by-step:

1. `apps/api/src/lra_api/main.py` creates `FastAPI(...)`, sets CORS, includes `api_router` at `/api/v1`, includes `/metrics`.
2. On startup, `startup_event()` calls `MinioStorageService.ensure_buckets()` unless `APP_ENV=testing`.
3. Every request passes through `metrics_middleware()`: - calls `RateLimiter.enforce(request)`. - then increments Prometheus `REQUEST_COUNTER`.
4. Protected endpoints depend on `get_current_user()` in `api/deps.py`: - reads bearer token via `OAuth2PasswordBearer`. - decodes token with `decode_token(token, expected_type="access")`. - fetches `User` by `payload["sub"]`.

Auth payload shapes (`schemas/auth.py`):

- `POST /api/v1/auth/login` request:
- `{ "email": "user@example.com", "password": "..." }`
- `POST /api/v1/auth/login` response (`TokenResponse`):
- `{ "access_token": "...", "refresh_token": "...", "token_type": "bearer", "expires_in_seconds": 1800 }`

3.2 Notebook and Session Lifecycle

Notebook flow:

1. Create notebook via `POST /api/v1/notebooks`.
2. Handler `create_notebook()` persists `Notebook(owner_id, title, description, tags)`.
3. Notebook id is used for document ingestion and chat sessions.

Chat session flow:

1. `POST /api/v1/chat/sessions` with `ChatCreateRequest` (`notebook_id`, `title`) creates `Chat`.
2. `GET /api/v1/chat/sessions/{chat_id}` returns session metadata and message history.
3. Message history output includes `role`, `content`, `citations`, `created_at`.

Conversation memory persistence:

- Stored in relational tables:
- `chats`
- `messages` (`role`, `content`, `citations_json`)
- `citations` (normalized citation rows linked to assistant message)

3.3 Upload and Ingestion Queue Flow

Entry endpoint:

- `POST /api/v1/documents/upload/{notebook_id}` (`documents.py`).

Execution path:

```
validate_upload(file.filename, content)
document = await ingestion.ingest_upload(...)
task = process_document_upload.delay(str(document.id))
return IngestionJobResponse(job_id=task.id, document_id=document.id, status="queued")
```

Important behavior:

- Ownership check: notebook must belong to authenticated user.
- Error handling:
- empty file -> `400` (`"Empty file"`)
- unsupported extension -> `400` from `validate_upload`
- oversize file (> 50 MB) -> `400`
- Upload validation rules are in `utils/uploads.py`:
- allowed extensions include PDF, DOCX, TXT, CSV, XLS/XLSX, PPTX, PNG/JPG/WEBP/BMP.

Job status:

- `GET /api/v1/documents/jobs/{job_id}` returns Celery `AsyncResult.status`.

3.4 External Source Ingestion Flows

All three endpoints use `ImportRequest`:

- `{ "notebook_id": "<uuid>", "source_url": "<url>" }`

Website:

- `POST /api/v1/documents/import/website`
- `ConnectorService.fetch_website_text(url)`:
- fetches HTML via `httpx`.
- extracts text via BeautifulSoup.

GitHub:

- `POST /api/v1/documents/import/github`
- `ConnectorService.fetch_github_repo_snapshot(url)`:
- shallow clone with `Repo.clone_from(..., depth=1)`.
- reads `*.md`, `*.txt`, fallback `README*`.
- concatenates into one markdown-like text payload.

YouTube transcript:

- `POST /api/v1/documents/import/youtube`
- `ConnectorService.fetch_youtube_transcript(url)`:
- runs `yt-dlp` with auto subtitles in English.
- converts `.vtt` to plain text via `_vtt_to_text`.

3.5 Parse -> OCR Fallback -> Chunk -> Embed -> Index

Worker entrypoint:

- `process_document_upload(document_id)` in `workers/tasks.py`.
- Calls `IngestionService.process_document(db, document_id)`.

Detailed pipeline (`services/ingestion/pipeline.py`):

1. Mark `document.indexing_status = "processing"`.
2. Fetch raw bytes from MinIO by `storage_key`.
3. Parse text with `parse_bytes(content, filename, mime_type)`.
4. If parsed text is empty and `_requires_ocr(...)` is true: - call `_run_ocr_with_timeout(...)` -> `OllamaClient.ocr_image`. - cap OCR output to 4000 chars. - fallback to placeholder text if OCR still empty.
5. Chunk text with `chunk_text(text, chunk_size=1000, overlap=150)`.
6. For each chunk in `_persist_chunks`: - embed text using `OllamaClient.embed`. - persist `DocumentChunk(document_id, chunk_index, text, token_count, metadata_json={})`. - collect Qdrant `PointStruct` payload:
    - `document_id`, `chunk_id`, `text`, `source_name`.
7. Ensure Qdrant collection exists, then upsert points.
8. Extract graph relations and upsert into Neo4j via `GraphService.upsert_document_graph`.
9. Mark `document.indexing_status = "indexed"` and store metadata: - `chunks` - `graph_edges` - `source_extension`

3.6 Hybrid Search Flow (Semantic + Lexical + Rerank)

Endpoint:

- `POST /api/v1/search/query`

Request shape (`SearchRequest`):

- `query: str` (required, min 2 chars)
- `notebook_id: UUID | null`
- `top_k: int` default 8
- `metadata_filters: dict` (currently accepted by schema but not used in retrieval service call)
- `source_types: list[str]`
- `date_from`, `date_to`

Search execution (`RetrievalService.search`):

1. Semantic search: - embed query with `OllamaClient.embed`. - query Qdrant collection.
2. Lexical search: - SQL over `document_chunks` + `documents`. - uses Postgres full-text search with `ts_rank_cd`.
3. Merge scores by `chunk_id`.
4. `_filter_existing_chunks(...)` removes stale vector hits by checking live relational rows and ownership filters.
5. Optional rerank with `sentence_transformers.CrossEncoder` when `RERANKER_ENABLED=true`.
6. Return list of `SearchHit`.

Response shape (`SearchResponse`):

- `{ "query": "...", "hits": [ { "chunk_id", "document_id", "score", "lexical_score", "semantic_score", "text", "source_name", "page_number", "metadata" } ] }`

Persistence side effect:

- `search_history` row is inserted with `user_id`, `notebook_id`, `query`, `result_count`.

3.7 Grounded QA with Citations and Chat Memory

Two modes:

- One-shot RAG: `POST /api/v1/chat/rag`
- Session-based chat: `POST /api/v1/chat/sessions/{chat_id}/messages`

Session message flow (`send_message`):

1. Verify chat belongs to current user via notebook join.
2. Save user message to `messages` table (`role="user"`).
3. Run retrieval with notebook scope.
4. Build answer using `RetrievalService.build_answer(query, hits)`.
5. Save assistant message with `citations_json`.
6. Save normalized citation rows in `citations` table.
7. Return `ChatMessageResponse`: - `answer: str` - `citations: list[CitationPayload]`

Citation payload shape:

- `document_id: str`
- `chunk_id: str`
- `quote: str`
- `source_name: str`
- `page_number: int | null`

Streaming mode (`/stream`):

- Emits SSE event sequence:
- `start` with JSON citation list
- repeated `token` events (split by whitespace)
- `done`

3.8 Research Analysis Flow

Endpoints in `research.py`:

- `POST /api/v1/research/summary`
- `POST /api/v1/research/compare`
- `POST /api/v1/research/timeline`
- `POST /api/v1/research/glossary`
- `POST /api/v1/research/entities`

Service:

- `ResearchService` methods call `OllamaClient.generate` with task-specific prompts.

Input shapes:

- `SummaryRequest`: `{ "text": "...", "mode": "executive" }` (`mode` optional, default `executive`)
- `CompareRequest`: `{ "text_a": "...", "text_b": "..." }`

Output shape:

- `ResearchOutput`: `{ "content": "..." }`

Document comparison is implemented as `ResearchService.compare`.

3.9 Study Generation Flow (Guide + Flashcards + Quiz)

Endpoint:

- `POST /api/v1/study/generate`

Request (`StudyGenerateRequest`):

- `notebook_id: UUID`
- `topic: str`
- `difficulty: "beginner" | "intermediate" | "advanced"`

Execution:

1. Check notebook ownership.
2. `StudyService.generate(notebook_id, topic, difficulty)` asks Ollama to return JSON with keys: - `study_guide` - `flashcards` - `quiz`
3. If JSON parse fails, fallback stores raw text as `study_guide` with empty lists.

4. Persist: - `StudyMaterial(kind="study_bundle", title=...)` - `Flashcard` rows for each card - one `Quiz` row with `questions_json`

Retrieval endpoints:

- `GET /api/v1/study/flashcards/{notebook_id}`
- `GET /api/v1/study/quizzes/{notebook_id}`

3.10 Knowledge Graph Flow

Ingestion-time graph enrichment:

- `IngestionService.process_document` calls `GraphService.upsert_document_graph(document_id, text)`.

Extraction model:

- heuristic capitalized-token extractor (`extract_relations`) builds `EntityRelation(source, relation, target)`.
- relations are stored in Neo4j as `(:Entity)-[:RELATED_TO {document_id, relation}]->(:Entity)`.

Query endpoint:

- `POST /api/v1/graph/query`
- request: `{ "query": "AI", "limit": 50 }`
- response: `{ "nodes": [...], "edges": [...] }`

3.11 Monitoring and Health Flow

Health:

- `GET /api/v1/health` checks:
- DB with `SELECT 1`
- Redis `ping`
- Ollama `/api/tags` through client
- returns `HealthResponse`:
- `status`, `database`, `redis`, `ollama`

System:

- `GET /api/v1/system/models` -> list installed Ollama models.
- `GET /api/v1/system/status` -> CPU/memory/GPU/queue metrics.

Metrics:

- `GET /metrics` returns Prometheus format.
- `REQUEST_COUNTER` and `RETRIEVAL_LATENCY` are defined in `services/monitoring/metrics.py`.

3.12 Frontend Flow Mapping

Implemented UI-to-API mappings:

- `LoginPage.tsx` -> `POST /auth/login`
- `NotebooksPage.tsx` -> `POST /notebooks`
- `DocumentsPage.tsx` -> `POST /documents/upload/{notebook_id}`
- `SearchPage.tsx` -> `POST /search/query`
- `ChatPage.tsx` -> `POST /chat/sessions`, then `/chat/sessions/{chat_id}/stream`
- `StudyToolsPage.tsx` -> `POST /study/generate`
- `FlashcardsPage.tsx` -> `GET /study/flashcards/{notebook_id}`
- `QuizzesPage.tsx` -> `GET /study/quizzes/{notebook_id}`
- `KnowledgeGraphPage.tsx` -> `POST /graph/query`
- `ModelManagerPage.tsx` -> `GET /system/models`
- `MonitoringPage.tsx` -> `GET /system/status`

Current UI gaps relative to backend:

- No dedicated frontend forms for `/documents/import/website`, `/documents/import/github`, `/documents/import/youtube`.
- OCR page is informational; it does not call `POST /documents/ocr`.
- Knowledge Base and Workspace pages are mostly placeholders for future richer explorer/session UX.
- No mind-map endpoint or page found in backend/frontend.

Module 4: Setup & Run Guide

This module lists real setup/run instructions found in `README.md`, docs, manifests, and scripts.

4.1 Prerequisites

- Python `>=3.12,<3.13` (from `pyproject.toml`)
- `uv` package manager
- Node.js + npm (frontend uses Vite + React)
- Docker + Docker Compose
- Local Ollama server for model inference
- `yt-dlp` available for YouTube transcript ingestion path

4.2 Environment and Dependencies

1. Copy environment template:

```
cp .env.example .env
```

2. Create and sync Python environment:

```
uv venv --python 3.12
source .venv/bin/activate
uv sync --group dev
```

3. Install frontend dependencies:

```
cd apps/web
npm install
cd ../..
```

4.3 Required Environment Variables

From `.env.example`, required groups are:

- App:
- `APP_ENV`, `APP_NAME`, `APP_HOST`, `APP_PORT`, `APP_SECRET_KEY`, `APP_ACCESS_TOKEN_EXPIRE_MINUTES`, `APP_REFRESH_TOKEN_EXPIRE_DAYS`, `APP_ALLOWED_ORIGINS`
- Relational/cache:
- `DATABASE_URL`, `SYNC_DATABASE_URL`, `REDIS_URL`
- Vector DB:
- `QDRANT_URL`, `QDRANT_API_KEY`, `QDRANT_COLLECTION`
- Graph DB:
- `NEO4J_URI`, `NEO4J_USER`, `NEO4J_PASSWORD`
- Object storage:
- `MINIO_ENDPOINT`, `MINIO_ACCESS_KEY`, `MINIO_SECRET_KEY`, `MINIO_SECURE`, `MINIO_BUCKET_RAW`, `MINIO_BUCKET_DERIVED`
- Ollama/model routing:
- `OLLAMA_BASE_URL`, `OLLAMA_CHAT_MODEL`, `OLLAMA_SUMMARY_MODEL`, `OLLAMA_LIGHT_MODEL`, `OLLAMA_FALLBACK_MODEL`, `OLLAMA_EMBEDDING_MODEL`, `OLLAMA_OCR_MODEL`, `OLLAMA_TRANSLATION_MODEL`, `OLLAMA_REQUEST_TIMEOUT`
- Retrieval:
- `RERANKER_MODEL`, `RERANKER_ENABLED`
- Worker:
- `CELERY_BROKER_URL`, `CELERY_RESULT_BACKEND`
- Monitoring:
- `PROMETHEUS_ENABLED`, `ENABLE_GPU_METRICS`

4.4 Infrastructure Startup

Start all dependencies:

```
docker compose up -d postgres redis qdrant neo4j minio prometheus grafana
```

Or full stack including app containers:

```
docker compose up -d
```

4.5 Database Migration and First User Bootstrap

Run migrations:

```
source .venv/bin/activate
uv run alembic -c apps/api/alembic.ini upgrade head
```

Bootstrap admin (one-time):

Option A (script):

```
./infra/scripts/bootstrap_admin.sh admin@example.com 'StrongPassword123!'
```

Option B (direct API):

```
curl -X POST "http://localhost:18000/api/v1/auth/bootstrap" \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@example.com","password":"StrongPassword123!"}'
```

4.6 Run API, Worker, Frontend

API:

```
source .venv/bin/activate
uv run uvicorn lra_api.main:app --host 0.0.0.0 --port 18000 --reload
```

Worker:

```
source .venv/bin/activate
uv run celery -A lra_api.workers.celery_app.celery_app worker -l info
```

Frontend:

```
cd apps/web
npm run dev
```

Frontend API base URL:

- default in code: `http://localhost:18000/api/v1`
- override with `VITE_API_BASE_URL` if needed.

4.7 Core Command Sequences for Main Features

After login, use bearer token in `Authorization: Bearer <TOKEN>`.

Notebook creation:

```
curl -X POST "http://localhost:18000/api/v1/notebooks" \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"title":"My Notebook","tags":[]}'
```

Upload ingestion:

```
curl -X POST "http://localhost:18000/api/v1/documents/upload/<NOTEBOOK_ID>" \
  -H "Authorization: Bearer <TOKEN>" \
  -F "file=@/path/to/file.pdf"
```

Website/GitHub/YouTube ingestion:

```
curl -X POST "http://localhost:18000/api/v1/documents/import/website" \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"notebook_id":"<NOTEBOOK_ID>","source_url":"https://example.com"}'

curl -X POST "http://localhost:18000/api/v1/documents/import/github" \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"notebook_id":"<NOTEBOOK_ID>","source_url":"https://github.com/<owner>/<repo>"}'
```

```
curl -X POST "http://localhost:18000/api/v1/documents/import/youtube" \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"notebook_id":"<NOTEBOOK_ID>","source_url":"https://www.youtube.com/watch?v=<id>"}'
```

OCR endpoint:

```
curl -X POST "http://localhost:18000/api/v1/documents/ocr" \
  -H "Authorization: Bearer <TOKEN>" \
  -F "file=@/path/to/image.png"
```

Search:

```
curl -X POST "http://localhost:18000/api/v1/search/query" \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"query":"hybrid retrieval","notebook_id":"<NOTEBOOK_ID>","top_k":8}'
```

Chat:

```
curl -X POST "http://localhost:18000/api/v1/chat/sessions" \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"notebook_id":"<NOTEBOOK_ID>","title":"Research Session"}'

curl -X POST "http://localhost:18000/api/v1/chat/sessions/<CHAT_ID>/messages" \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"query":"What does the corpus say about X?","top_k":6}'
```

Study generation:

```
curl -X POST "http://localhost:18000/api/v1/study/generate" \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"notebook_id":"<NOTEBOOK_ID>","topic":"X","difficulty":"intermediate"}'
```

4.8 Database/External-Service Migration or Seeding Notes

- Relational schema migration is Alembic-based (`apps/api/alembic/`).
- There is no separate data seeding script besides admin bootstrap.
- Qdrant collections are created lazily in ingestion when first embeddings are persisted (`_ensure_qdrant_collection`).
- MinIO buckets are ensured at startup and ingestion time.
- Neo4j relations are inserted during ingestion (`upsert_document_graph`).

4.9 PDF Export of This Handbook

Example pandoc command:

```
pandoc docs/guides/ZERO_TO_HERO_STUDY_HANDBOOK.md -o docs/guides/ZERO_TO_HERO_STUDY_HANDBOOK.pdf
```

Module 5: Study Plan & Practice Exercises

5.1 Ordered Study Plan

Recommended order for a new learner:

1. Read runtime contracts first: - `.env.example` - `pyproject.toml` - `docker-compose.yml`
2. Understand app assembly and auth: - `main.py` , `api/router.py` , `api/deps.py` , `core/security.py`
3. Learn data model and persistence: - `db/models/entities.py` - `alembic/versions/20260628_0001_initial_schema.py`
4. Deep dive ingestion: - `services/ingestion/parsers.py` - `services/ingestion/chunking.py` - `services/ingestion/pipeline.py` - `workers/tasks.py`
5. Deep dive retrieval and QA: - `services/retrieval/service.py` - `api/v1/endpoints/search.py` - `api/v1/endpoints/chat.py`

6. Study feature extensions: - `services/research/service.py` - `services/study/service.py` - `services/graph/service.py`
7. Map frontend to backend: - `apps/web/src/services/api.ts` - `apps/web/src/app/router.tsx` - key pages (`DocumentsPage`, `SearchPage`, `ChatPage`, `StudyToolsPage`)
8. Read docs after code: - `docs/api/API.md` - `docs/architecture/ARCHITECTURE.md` - selected guides (`INSTALLATION`, `RAG`, `OCR`, `SEARCH`)

5.2 Practice Exercises (with Model Answer Outlines)

Exercise 1:

- Task: Trace the full path for file upload from HTTP request to vector index insertion.
- Target files: `endpoints/documents.py`, `workers/tasks.py`, `services/ingestion/pipeline.py`.

Model answer outline:

- `upload_document` validates + saves metadata (`ingest_upload`) + queues Celery task.
- `process_document_upload` invokes `process_document`.
- `process_document` parses/OCR/chunks and `_persist_chunks` writes DB chunks + Qdrant points.

Exercise 2:

- Task: Explain exactly when OCR is triggered and what fallback behavior exists.
- Target files: `parsers.py`, `pipeline.py`, `ollama/client.py`.

Model answer outline:

- Parser returns empty string for image-like content.
- `process_document` checks empty parsed text and `_requires_ocr`.
- OCR uses `ocr_image`; timeout bounded; output capped to 4000 chars; placeholder text if still empty.

Exercise 3:

- Task: Explain how hybrid search scores are created and merged.
- Target files: `services/retrieval/service.py`.

Model answer outline:

- Semantic score from Qdrant vector similarity.
- Lexical score from `ts_rank_cd`.
- Merge by `chunk_id`; combine scores; optional cross-encoder rerank.
- Filter stale chunk ids against relational DB before final output.

Exercise 4:

- Task: Show how conversation memory is persisted and retrieved.
- Target files: `endpoints/chat.py`, `db/models/entities.py`.

Model answer outline:

- User and assistant messages are inserted into `messages`.
- Assistant citations stored in `messages.citations_json` and normalized `citations`.
- Session history retrieved with `GET /chat/sessions/{chat_id}` ordered by `created_at`.

Exercise 5:

- Task: Compare one-shot RAG (`/chat/rag`) vs session chat (`/chat/sessions/{chat_id}/messages`).
- Target files: `endpoints/chat.py`.

Model answer outline:

- One-shot returns answer/citations without persisting chat/message rows.
- Session mode persists user and assistant messages and citation rows.

Exercise 6:

- Task: Identify and explain upload error cases and where each is enforced.
- Target files: `utils/uploads.py`, `endpoints/documents.py`.

Model answer outline:

- Empty body check in endpoint -> `400`.
- Unsupported extension and >50MB checks in `validate_upload` -> `ValueError` -> `400`.
- Notebook ownership missing -> `404`.

Exercise 7:

- Task: Explain how study output becomes relational data.
- Target files: `services/study/service.py`, `endpoints/study.py`, `entities.py`.

Model answer outline:

- Service returns `StudyResponse` from model JSON.
- Endpoint creates `StudyMaterial`, multiple `Flashcard` rows, and one `Quiz` row with `questions_json`.

Exercise 8:

- Task: Check whether mind map generation is implemented.
- Target files: `api/router.py`, `api/v1/endpoints/*`, `services/*`, frontend pages.

Model answer outline:

- No `/mind-map` or equivalent endpoint/service/page exists in scanned code.
- Current advanced outputs include summary/compare/timeline/glossary/entities, study tools, and graph query.

Exercise 9:

- Task: Map frontend pages to real backend endpoints and list missing UI coverage.
- Target files: `apps/web/src/pages/*.tsx`, `apps/web/src/services/api.ts`, backend endpoint files.

Model answer outline:

- Implemented mappings: login/upload/search/chat/study/flashcards/quizzes/graph/models/status.
- Missing direct UI flows: website/github/youtube import forms, OCR POST form, richer document explorer/knowledge operations.

5.3 Learner Verification Checklist

Use this checklist to confirm mastery:

- Can you explain how `main.py` wires middleware, routers, metrics, and startup hooks?
- Can you trace a document from upload/import to `indexing_status="indexed"`?
- Can you explain parser-first + OCR-fallback logic with exact triggering conditions?
- Can you describe Qdrant payload fields and where they are created?
- Can you explain hybrid retrieval internals, including lexical SQL and score merging?
- Can you explain how stale vector hits are filtered with relational checks?
- Can you describe the exact citation payload shape returned by chat/RAG?
- Can you explain how chat session memory is stored and resumed?
- Can you explain how study artifacts are generated and persisted to `study_materials`, `flashcards`, and `quizzes`?
- Can you explain how graph edges are extracted and queried in Neo4j?
- Can you list required `.env` keys by subsystem and their purpose?
- Can you identify currently implemented frontend flows versus placeholder/missing ones?
- Can you explain why mind map generation is currently not part of this codebase?